



ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ

**BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT**

(Autonomous Institution Under VTU)

Yelahanka, Bengaluru -560119

# Department of Artificial Intelligence & Machine Learning



## OPERATING SYSTEMS BCS303

### Third Semester Laboratory Manual AY:2025-26

Designed By:  
Course Coordinators

### **Institute Vision**

To emerge as one of the finest technical institutions of higher learning, to develop **engineering professionals** who are technically competent, ethical and environment friendly for **betterment of the society**.

### **Institute Mission**

Accomplish stimulating learning environment through **high quality academic instruction, innovation and industry-institute interface**.

### **Department Vision**

To emerge as a leading department in AI and ML by preparing **skilled, responsible, and eco-friendly professionals** who use technology to improve society.

### **Department Mission**

- To equip students with in-depth knowledge in Artificial Intelligence and Machine Learning, built upon a robust foundation in Computer Science and Engineering through **industry focused curriculum with practical learning**.
- To **encourage research and innovation through industry partnerships** and sustainable technology practices.
- To inspire students toward ethical leadership and entrepreneurship through **innovative and collaborative student-led activities**.

**Semester:**           **III**

**Course Code:**       **BCS303**

**Course Name:**       **OPERATING SYSTEMS**

| <b>Course Coordinators</b> | <b>Signature</b> | <b>Date</b> |
|----------------------------|------------------|-------------|
| <b>Dr. Sowmya V L</b>      |                  |             |
| <b>Prof Pavan Mulgund</b>  |                  |             |

|

**Head of Department (Sign & Date)**



| <b>PRACTICAL COMPONENT OF IPCC</b>                    |                                                                                                                                                                                                                                                        |
|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PART A</b>                                         |                                                                                                                                                                                                                                                        |
| <b>INTRODUCTORY SECTION – CORE OS FUNCTIONALITIES</b> |                                                                                                                                                                                                                                                        |
| 1                                                     | Write a C program to demonstrate open (), read (), write () and close () system call.                                                                                                                                                                  |
| 2                                                     | Create the following types of files using shell commands and system calls<br>a. soft link (symlink system call)<br>b. hard link (link system call)                                                                                                     |
| 3                                                     | Write a C program to simulate the process system calls (fork (), exec () and wait (), create process () and terminate process())                                                                                                                       |
| <b>PART – B</b>                                       |                                                                                                                                                                                                                                                        |
| <b>Shell Scripts</b>                                  |                                                                                                                                                                                                                                                        |
| 4                                                     | Write a shell program which takes two file names as arguments, if their contents are the same then remove the second file.                                                                                                                             |
| 5                                                     | Write a shell program that takes a command line argument and reports on whether it is directory, a file, or something else.                                                                                                                            |
| 6                                                     | Write a shell script that accepts two file names as arguments, checks if the permissions for these files are identical and if the permissions are identical, output common permission and otherwise output each file name followed by its permissions. |
| 7                                                     | Write a shell program to prints the arguments in reverse order                                                                                                                                                                                         |
| 8                                                     | Write a shell program to print the prime numbers between the specified range                                                                                                                                                                           |
| <b>PART – C</b>                                       |                                                                                                                                                                                                                                                        |
| <b>API based programs</b>                             |                                                                                                                                                                                                                                                        |
| 1.                                                    | Write a C program to implement UNIX ln, mv, rm commands using APIs                                                                                                                                                                                     |
| 2.                                                    | Write a program in C to display the contents of a named file on standard output device., Also Write a program to copy the contents of one file to another.                                                                                             |
| 3.                                                    | Write a C program that reads every 100th byte from the file, where the file name is given as command line argument                                                                                                                                     |
| 4.                                                    | Write a C program to create a process by using fork () and vfork() system call                                                                                                                                                                         |
| 5.                                                    | Write a C program to demonstrate the process is Zombie and to avoid the Zombie process.                                                                                                                                                                |
| 6.                                                    | Write a C program to create an Orphan Process                                                                                                                                                                                                          |
| 7.                                                    | Write a C program to demonstrate a parent process that uses wait ( ) system call to catch the child 's exit code.                                                                                                                                      |
| 8.                                                    | Write a Program to demonstrate race condition.                                                                                                                                                                                                         |
| 9.                                                    | Write a program to create, writes to, and reads from a pipe. Also Write a program to create a pipe from the parent to child and send data down the pipe.                                                                                               |

|     |                                                                                                                                                |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------|
| 10. | Write a program to create a semaphore and initialize value to the semaphore.<br>a. Create a binary semaphore<br>b. Create a counting semaphore |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------|

**Course Outcomes:**

The students will be able to:

CO1: Describe the basics of the operating systems services, process, threads and communication

CO2: Apply appropriate scheduling methods for process execution and its management

CO3: Illustrate suitable methods to handle deadlocks and process synchronization

CO4: Analyze memory management techniques and its allocation policies

CO5: Discuss various Implementation of File systems, security mechanism with respect to different storage management technologies

CO6: Apply APIs related to Process, Files and IPC to develop operating system applications.

**Activity Based Learning (Suggested Activities in Class)/ Practical Based learning**

Assessment Methods

- Gate Based Aptitude Questions
- Lab Assessment (30 Marks)

**Continuous Comprehensive Assessments (CCA):**

**Total Marks 10 scaled down to 5 Marks**

| CCA                            | Marks allotted | Criteria                                                            | Good                                                                                                               | Average                                                                         | Worst                                                       |
|--------------------------------|----------------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------|
| GATE Based Aptitude Test (CCA) | 10             | Accuracy of Answers<br>( 5 marks)                                   | 80% or more answers correct.<br>(5-4 marks)                                                                        | 50–79% answers correct<br>( 3-2marks)                                           | Less than 50% correct<br>( 1-0 Mark)                        |
|                                |                | Pro Problem-Solving Approach<br>blem-Solving Approach<br>( 5 marks) | Logical and systematic method, clear step-by-step working shown for numerical/analytical questions.<br>(5-4 marks) | Partial logical flow, some steps missing, but reasoning visible.<br>(3-2 marks) | Random guesses, unclear or no working steps.<br>(1-0 marks) |

### Rubrics for Continuous Internal Evaluation (CIE) :30 Marks (Scale down to 15)

|                                                 | <b>Excellent</b>                                                                                                                                           | <b>Very Good</b>                                                                                                        | <b>Good</b>                                                                              | <b>Satisfactory</b>                                                              |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Fundamental Knowledge (4) (PO1)</b>          | The student has well depth knowledge of the topics related to the course (4)                                                                               | Student has good knowledge of some of the topics related to course (3)                                                  | Student is capable of narrating the answer but not capable to show in depth knowledge(2) | Student has not understood the concepts clearly (1)                              |
| <b>Design Of Experiment (5) (PO2 &amp; PO3)</b> | Student is capable of discussing more than one design for his/her problem statement and capable of proving the best suitable design with proper reason (5) | Student is capable of discussing few designs for his/her problem statement but not capable of selecting best(4)         | Student is capable of discussing single design with its merits and de-merits(3)          | Student is capable of explaining the design (1-2)                                |
| <b>Implementation (8) (PO3 &amp;PO8)</b>        | Student is capable of implementing the design with best suitable algorithm considering optimal solution. (7-8)                                             | Student is capable of implementing the design with best suitable algorithm and should be capable of explaining it (5-6) | Student is capable of implementing the design with proper explanation.(3-4)              | Student is capable of implementing the design. (1-2)                             |
| <b>Result &amp;Analysis (5) (PO4)</b>           | Student is able to run the program on various cases and compare the result with proper analysis. (5)                                                       | Student will be able to run the program for all the cases.(4)                                                           | Student will be able to run the code for few cases and analyze the output.(3)            | Student will be able to run the program but not able to analyze the output.(1-2) |
| <b>Demonstration (8) (PO9)</b>                  | The lab record is well-organized, with clear sections (e.g., Introduction, Method, Results,                                                                | The lab record is organized, with clear sections, but some sections                                                     | The lab record lacks clear organization or structure. Some sections are                  | The lab record is poorly organized, with missing                                 |

|  |                                                                      |                                 |                                     |                               |
|--|----------------------------------------------------------------------|---------------------------------|-------------------------------------|-------------------------------|
|  | Conclusion).<br>Transitions<br>between sections<br>are smooth. (7-8) | are not well-<br>defined. (5-6) | unclear or<br>incomplete. (3-<br>4) | or unclear<br>sections. (1-2) |
|--|----------------------------------------------------------------------|---------------------------------|-------------------------------------|-------------------------------|

**Rubrics for Continuous Internal Evaluation (CIE) Test :20 Marks (scale down to 10)**

| Sl.No | Parameters          | Marks<br>Allocated | High                                                                                                | Medium                                                                                                  | Low                                                                                                   |
|-------|---------------------|--------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 1     | Program<br>Write UP | 5                  | The student was able<br>to write the program<br>& Algorithm or<br>flowchart correctly<br><b>(5)</b> | The student was<br>able to write the<br>program &<br>Algorithm or<br>flowchart with<br>mistake<br>(2-4) | The student<br>was not able<br>to write the<br>program &<br>Algorithm<br>or flowchart<br><b>(0-1)</b> |
| 2     | Execution           | 10                 | Program executed<br>and showed the<br>output<br><b>(10 Marks)</b>                                   | Partially<br><b>(9-7)</b>                                                                               | Not shown<br><b>(0-3)</b>                                                                             |
| 3     | Viva Voce           | 5                  | Answered all the<br>questions related to<br>the program properly<br><b>(5)</b>                      | Partially<br>(2-4)                                                                                      | Not<br>answered<br><b>(0-1)</b>                                                                       |



| Program No.                                                   | Short Title                                                                                                  | Page No. |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|----------|
| <b>PART A – Introductory Section: Core OS Functionalities</b> |                                                                                                              |          |
| 1                                                             | Demonstration of open(), read(), write(), close() system calls                                               | 6        |
| 2                                                             | Creating hard link and soft (symbolic) link using link() and symlink()                                       | 9        |
| 3                                                             | Simulation of process system calls: fork(), exec(), wait(), create and terminate process                     | 13       |
| <b>PART B – Shell Scripts</b>                                 |                                                                                                              |          |
| 4                                                             | Shell script to compare two files and delete the second if contents are identical                            | 16       |
| 5                                                             | Shell script to check whether a command-line argument is a file, directory, or something else                | 18       |
| 6                                                             | Shell script to compare permissions of two files and display common / individual permissions                 | 20       |
| 7                                                             | Shell script to print all command-line arguments in reverse order                                            | 22       |
| 8                                                             | Shell script to print all prime numbers between a specified lower and upper limit                            | 24       |
| <b>PART C – API Based Programs</b>                            |                                                                                                              |          |
| 1                                                             | Implementing UNIX ln, mv, and rm commands using system calls (link(), rename(), unlink())                    | 26       |
| 2                                                             | Display contents of a file on standard output and copy contents from one file to another                     | 30       |
| 3                                                             | Reading every 100th byte from a file using fseek() and fgetc()                                               | 33       |
| 4                                                             | Creating processes using fork() and vfork() system calls                                                     | 35       |
| 5                                                             | Demonstrating zombie process creation and avoiding zombie using wait()                                       | 38       |
| 6                                                             | Creating an orphan process and observing change of parent PID (adopted by init / systemd)                    | 42       |
| 7                                                             | Parent process using wait() / waitpid() to catch child's exit code (multiple children with different status) | 44       |
| 8                                                             | Demonstration of race condition using fork() and unsynchronized writes to stdout                             | 47       |
| 9                                                             | Creating a pipe, writing to and reading from it; parent-to-child communication through a pipe                | 50       |
| 10                                                            | Creating and initializing semaphores: binary semaphore and counting semaphore using sem_init()               | 54       |

## PART-A

### **Program 1: Create a C program to demonstrate open(), read(), Write(),close() system calls.**

A system call is just what its name implies—a request for the operating system to do something on behalf of the user's program

1. **open()**
2. **read()**
3. **write()**
4. **close()**

#### **1. open()**

```
#include <fcntl.h>
```

```
int open( char *filename, int access, int permission );
```

The available access modes are

```
O_RDONLY O_WRONLY O_RDWR O_APPEND O_BINARY O_TEXT
```

The permissions are

```
S_IWRITE S_IREAD S_IWRITE | S_IREAD
```

The open() function returns an integer value, which is used to refer to the file. If unsuccessful, it returns -1, and sets the global variable errno to indicate the error type.

#### **2. read()**

```
#include <fcntl.h>
```

```
int read( int handle, void *buffer, int nbyte );
```

The read() function attempts to read nbytes from the file associated with the handle, and places the characters read into the buffer. If the file is opened using O\_TEXT, it removes carriage returns and detects the end of the file. The function returns the number of bytes read. On end-of-file, 0 is returned, on error it returns -1, setting errno to indicate the type of error that occurred.

#### **3. write()**

```
#include <fcntl.h>
```

```
int write( int handle, void *buffer, int nbyte );
```

The write() function attempts to write nbytes from the buffer to the file associated with handle. On text files, it expands each LF to a CR/LF. The function returns the number of bytes written to the file. A return value of -1 indicates an error, with errno set appropriately.

#### **4. close()**

```
#include <fcntl.h>
```

```
int close( int handle );
```

The close() function closes the file associated with the handle. The function returns 0 if successful, -1 to indicate an error, with errno set appropriately.

## 5. lseek()

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
off_t lseek(int fd, off_t offset, int whence);
```

- **fd:** This is the file descriptor of the open file whose offset is to be changed.
- **offset:** This is the number of bytes by which to move the file offset. It can be a positive or negative value.
- **whence:** This specifies the reference point for the offset. It can take one of the following values:
  - a. **SEEK\_SET:** The offset is set to offset bytes from the beginning of the file.
  - b. **SEEK\_CUR:** The offset is set to its current location plus offset bytes.
  - c. **SEEK\_END:** The offset is set to the size of the file plus offset bytes (i.e., offset bytes from the end of the file).

## Algorithm

1. **Start** the program.
2. **Declare variables:**
  - fd → file descriptor
  - buffer[80] → buffer for reading file data
  - message[] → data to write
  - nbytes → number of bytes read/written
3. **Open** the file "myfile.txt" in read/write mode.
  - Use O\_RDWR | O\_CREAT flags.
  - If opening fails, **print error and exit**.
4. **Print** message: "myfile.txt opened with read/write access".
5. **Write** the message to the file using write().
  - If writing fails, **print error, close file, and exit**.
6. **Move the file pointer** to the beginning of the file using lseek().
  - If seeking fails, **print error, close file, and exit**.
7. **Read** data from the file into buffer using read().
  - If reading fails, **print error, close file, and exit**.
8. **Null-terminate** the buffer: buffer[nbytes] = '\0'.
9. **Print** the data read from the file.
10. **Close** the file using close(fd).
11. **End** the program.

**Test cases:**

1. Check whether the program creates myfile.txt and writes the message correctly when the file does not exist.
2. Ensure that the program writes and reads the message properly when myfile.txt already exists and is empty.
3. Confirm that the program replaces the existing content of myfile.txt with the new message.
4. Observe how the program responds when myfile.txt has read-only permissions.
5. Check if the program successfully appends data when opened in append mode

**Program:**

*Implementation of open(), read(), write() and close() functions student@ubuntu:~\$ gedit os1.c*

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
int fd;
char buffer[80];
static char message[]="Welcome to Unix System Calls\n"; fd=open("myfile.txt",O_RDWR);
    if (fd != -1)
    {
        printf("myfile.txt opened with read/write access\n"); write(fd,message,sizeof(message));
        lseek(fd,0,0); read(fd,buffer,sizeof(message));
        printf("%s — was written to myfile.txt \n",buffer); close(fd);
    }
}
```

Note: First create empty file with myfile.txt before running the commands.

***Commands:***

```
student@ubuntu:~$ gedit myfile.txt
student@ubuntu:~$ gcc os1.c
student@ubuntu:~$ ./a.out
myfile.txt opened with read/write access
Welcome to Unix System Calls — was written to myfile.txt
student@ubuntu:~$ cat myfile.txt
Welcome to Unix System Calls student@ubuntu:~$
```

**Modifications /Extension of the Program**

1. Modify the open() call to create the file if it does not exist using O\_CREAT.
2. Set different File permission and check file attributes using ls-l

- 2. Create the following types of files using shell commands and system calls.**
- a. **Hardlink (link system call)**
  - b. **softlink (symlink system call)**

In **Unix/Linux operating systems**, **files** are represented by **inodes**, which store metadata (like file size, ownership, permissions, timestamps) and the **location of file data on disk**. The **filename** itself is a pointer (directory entry) to the inode. This design allows **multiple filenames to reference the same data** in different ways, which is implemented using **links**.

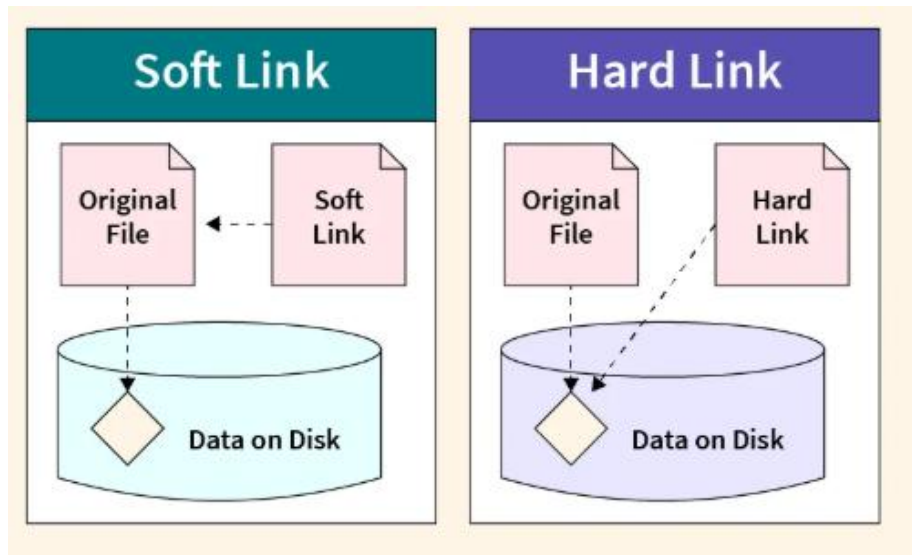
There are two types of links:

### **Hard Link**

- A **hard link** is a **direct reference to the inode** of a file.
- Multiple hard links share the **same inode number**, meaning they reference the **same physical data** on the disk.
- If the original file is deleted, the data remains accessible as long as at least **one hard link exists**.
- Limitations:
  - Cannot link directories (to prevent loops).
  - Cannot span across different filesystems.

### **Soft Link (Symbolic Link)**

- A **soft link**, or symbolic link, is a **special file containing the pathname of another file**.
- It has its own inode and does **not share data blocks** with the original file.
- If the original file is deleted, the soft link becomes **dangling**, i.e., it points to a non-existent file.
- Advantages:
  - Can link directories.
  - Can span different filesystems.



### System call for Hardlink

#### NAME

**link** - make a new name for a file

#### SYNTAX

```
#include <unistd.h>
```

```
int link(const char *oldpath, const char *newpath);
```

#### DESCRIPTION

**link()** creates a new link (also known as a hard link) to an existing file.

If *newpath* exists it will *not* be overwritten.

This new name may be used exactly as the old one for any operation; both names refer to the same file (and so have the same permissions and ownership) and it is impossible to tell which name was the 'original'.

`L = link(FILE1, FILE2)`, creates a hard link named FILE2 to an existing FILE1.

where, L is the value returned by the link() function.

#### RETURN VALUE

On success, zero is returned. On error, -1 is returned, and *errno* is set appropriately.

### System call for Softlink

#### NAME

**symlink** - make a new name for a file

```
#include <unistd.h>
```

```
int symlink(const char *oldpath, const char *newpath);
```

**DESCRIPTION**

**symlink()** creates a symbolic link named *newpath* which contains the string *oldpath*.

**RETURN VALUE**

On success, zero is returned. On error, -1 is returned, and *errno* is set appropriately.

**Shell Command to create a hard link is:**

```
$ ln [original filename] [link name]
```

Example:

```
$ ln source.txt final.txt
```

**Shell Command to create a soft link is:**

```
$ ln -s [original filename] [link name]
```

```
$ ln -s test.txt test1.txt
```

**Algorithm for creating a Hardlink :**

1. **Start** the program.
2. **Accept command-line arguments** — `argv[1]` as the existing file name and `argv[2]` as the new link name.
3. **Call the link() system call** with these two arguments.
4. Syntax: `link(const char *existing, const char *new);`
5. **Check the return value** of `link()`:  
If it returns 0, the hard link is created successfully.  
If it returns -1, an error occurred (e.g., file not found, permission denied).
6. **Display a success message** if the hard link is created.
7. **End the program.**

**Test cases:**

1. Check the program's ability to create a hard link for an existing file.
2. Observe the program's behavior when the source file does not exist.
3. Examine the output when the link file name already exists in the directory.
4. Check if the program can create multiple hard links pointing to the same source file.
5. Confirm that both files reflect identical content after the hard link is created.
6. Verify that deleting one of the linked files does not affect the data in the other.

**Algorithm for creating a softlink :**

1. **Start** the program.
2. **Accept command-line arguments:**  
`argv[1]` → name of the existing (target) file.  
`argv[2]` → name of the new symbolic link.
3. **Call the symlink() system call** with these two arguments to create the symbolic link.
4. **Check the return value** of `symlink()`:  
If it returns 0, the symbolic link is created successfully.  
If it returns -1, an error occurred (e.g., file not found, permission denied).
5. **Display a success message** when the symbolic link is created successfully.

6. **End** the program.

**Test cases:**

1. Check if the program successfully creates a symbolic link for an existing file.
2. Observe the program's behavior when the source file does not exist.
3. Examine the output when the symbolic link name already exists.
4. Test how the program responds when creating a link in a directory without write permissions.
5. Check if modifying the original file reflects changes when accessed through the symbolic link.
6. Confirm that deleting the original file leaves a broken symbolic link.
7. Verify that creating a symbolic link to a file in another directory works correctly.

**Program**

```
#include <stdio.h>
#include <unistd.h>
int main(int argc, char* argv[])
{
    // Link function
    int l = link(argv[1], argv[2]);
    // argv[1] is existing file name
    // argv[2] is link file name
    if (l == 0) {
        printf("Hard Link created successfully\n");
    }
    return 0;
}
```

**Output**

```
guest-Mojblw@ubuntu:~$ gedit sample.txt (type some content)
guest-Mojblw@ubuntu:~$ gcc hlink.c
guest-Mojblw@ubuntu:~$ ./a.out sample.txt sample1.txt
Hard Link created successfully
guest-Mojblw@ubuntu:~$ ls -l
total 44
```



### 3. Write a C program to simulate the process system calls (fork (), exec () and wait (), create process () and terminate process())

#### Algorithm:

1. **Start program**  
Print a message indicating simulation of process system calls.
2. **Call create\_process() function**
  - **Step 2.1:** Use fork() to create a new process.
    - If fork() returns **negative value** → error occurred → print error and exit.
    - If fork() returns **0** → this is the **child process**.
      - Print child PID and parent PID.
      - Print a message about replacing itself using exec().
      - Call execlp("ls", "ls", "-l", NULL) to replace the child with the ls -l program.
        - If exec() fails → print error and exit.
    - If fork() returns **positive value** → this is the **parent process**.
      - Print parent PID and child PID.
      - Call waitpid() to wait until the child finishes execution.
        - If child terminates normally → print exit status.
        - Else → print abnormal termination.
  - 3. **After child terminates**  
The parent resumes execution.  
Print message indicating return to parent process.
  - 4. **Call terminate\_process() function**
    - Print termination message with current process PID.
    - Call exit(0) to terminate.
  - 5. **End program** (control will not reach return 0 since exit() is called).

#### Test cases:

1. Check if the program successfully creates a child process using fork().
2. Observe whether the child process correctly executes the ls -l command using exec().
3. Check that the program prints the correct **PID** and **PPID** values for parent and child processes.
4. Observe the output when the exec() call fails (e.g., by changing "ls" to a non-existent command).
5. Verify that after the child terminates, the parent process resumes and executes terminate\_process().
6. Ensure the program terminates cleanly using the exit() system call.

#### Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // for fork(), exec(), getpid()
```

```
#include <sys/wait.h> // for wait()
// Function to simulate "create process"

void create_process() {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0) {
        // Child process
        printf("Child created successfully (PID: %d, PPID: %d)\n", getpid(), getppid());
        // Simulate exec() system call
        printf("Child process replacing itself using exec()...\n");
        execlp("ls", "ls", "-l", NULL); // replace with "ls -l"

        perror("exec failed"); // If exec fails, print error
        exit(1);
    }
    else {
        // Parent process
        printf("Parent (PID: %d) created a child (PID: %d)\n", getpid(), pid);
        // Simulate wait() system call
        int status;
        waitpid(pid, &status, 0);
        if (WIFEXITED(status)) {
            printf("Child terminated normally with exit status %d\n", WEXITSTATUS(status));
        } else {
            printf("Child terminated abnormally\n");
        }
    }
}

// Function to simulate "terminate process"
void terminate_process() {
    printf("Process (PID: %d) terminating...\n", getpid());
    exit(0);
}

int main()
{
    printf("Simulation of process system calls in C\n");
    // Simulate fork(), exec(), wait(), create_process(), terminate_process()
    create_process();
    // After child terminates, parent continues here
    printf("Back in parent process (PID: %d)\n", getpid());
    // Simulate terminate process
    terminate_process();
    return 0; // will not reach here since terminate_process() calls exit()
}
```

}

**Output:**

```
ubuntu@ubuntu2204:~/haarika$ gedit p3os.c
ubuntu@ubuntu2204:~/haarika$ gcc p3os.c
ubuntu@ubuntu2204:~/haarika$ ./a.out
Simulation of process system calls in C
Parent (PID: 4471) created a child (PID: 4472)
Child created successfully (PID: 4472, PPID: 4471)
Child process replacing itself using exec()...
total 60
-rwxrwxr-x 1 ubuntu ubuntu 16424 Sep 26 11:00 a.out
-rw-rw-r-- 1 ubuntu ubuntu 8 Sep 26 02:35 f10.txt
-rwxr-xr-x 1 ubuntu ubuntu 228 Sep 24 00:03 f5.txt
-rw-rw-r-- 1 ubuntu ubuntu 18 Sep 26 02:12 f7.txt
-rw-rw-r-- 1 ubuntu ubuntu 16 Sep 26 02:38 f8.txt
-rw-rw-r-- 1 ubuntu ubuntu 330 Sep 26 02:04 hlink.c
-rw-rw-r-- 2 ubuntu ubuntu 10 Sep 26 02:03 link1.txt
-rw-rw-r-- 1 ubuntu ubuntu 10 Sep 26 02:07 link3.txt
lrwxrwxrwx 1 ubuntu ubuntu 9 Sep 26 02:08 link4.txt -> link3.txt
-rw-rw-r-- 2 ubuntu ubuntu 10 Sep 26 02:03 link.txt
-rw-rw-r-- 1 ubuntu ubuntu 1680 Sep 26 10:59 p3os.c
-rw-rw-r-- 1 ubuntu ubuntu 348 Sep 26 02:06 symlink.c
lrwxrwxrwx 1 ubuntu ubuntu 8 Sep 26 02:25 test1.txt -> test.txt
Child terminated normally with exit status 0
Back in parent process (PID: 4471)
Process (PID: 4471) terminating...
```

**PART-B****4. Write a shell program which takes two file name as arguments if their contents are the same then remove the second file**

The **cmp** command in Linux/Unix is a utility used to compare two files byte by byte and determine whether they are identical or different. Its primary function is to check the contents of two files precisely by comparing each byte.

**Syntax:**

cmp option file1 file2

**Exit status:**

**0**-if files are identical

**1**-if file differ

**Greater than 1** if an error occurs

**Positional Parameters**

**\$?**-The exit status of the last command executed.

**\$0**-The filename of the current script.

**##**-The number of arguments supplied to a script.

**\$\$**-The process number of the current shell. For shell scripts, this is the process ID under which they are executing.

**Algorithm:**

**Step 1:** Start

**Step 2:** Input file1, file2

**Step 3:** Check the number of input arguments

    If the number of arguments  $\neq$  2, then

        Print "Usage: program file1 file2"

        Exit the program

    End If

**Step 4:** Read the contents of file1 and file2

**Step 5:** Compare the contents of the two files

    If contents of file1 = contents of file2, then

        Delete file2

        Print "Files are the same. file2 has been removed."

    Else

        Print "Files are different. No files removed."

        Display the contents of file1

        Display the contents of file2

    End If

**Step 6:** Stop

**Test Case1:**

1. Create 2 files a.txt b.txt with the content " welcome to Unix"

2. list the files in the directory.
3. Run the script and observe the output.

### **Test Case2:**

1. Modify the content of b.txt with “ welcome to OS”
2. Run the script and observe the output.
3. List the files in the directory

### **Program:**

```
clear
#Compare Two files , contents are same result is Zero or else 1
#Error redirection
cmp $1 $2 2>>Err
if [ $? -eq 0 ]
then
    echo Files are identical. the file $2 is removed.
rm $2
else
    echo Files are not identical. No action performed.
    echo Contents of file $1 are:
cat $1
    echo Contents of file $2 are:
cat $2
```

### **Output:**

```
sowmya@sowmya-HP-Notebook: ~$ cd /home/sowmya/os_practice
sowmya@sowmya-HP-Notebook: /home/sowmya/os_practice$ cat > fileA.txt
This is Operating systems Lab.
This is file A.
^C
sowmya@sowmya-HP-Notebook: /home/sowmya/os_practice$ cat > fileB.txt
This is file A.
^C
sowmya@sowmya-HP-Notebook: /home/sowmya/os_practice$ nano compare_files.sh
#Compare Two files , contents are same result is Zero or else 1
#Error redirection
cmp $1 $2 2>>Err
if [ $? -eq 0 ]
then
    echo "Files are identical. The file $2 is removed."
rm $2
else
    echo "Files are not identical. No action performed."
    echo "Contents of file $1 are:"
cat $1
    echo "Contents of file $2 are:"
cat $2
fi
^X
sowmya@sowmya-HP-Notebook: /home/sowmya/os_practice$ chmod +x compare_files.sh

=====
CASE 1 : WHEN BOTH FILES ARE IDENTICAL
=====

sowmya@sowmya-HP-Notebook: /home/sowmya/os_practice$ ./compare_files.sh fileA.txt fileB.txt
Files are identical. The file fileB.txt is removed.

sowmya@sowmya-HP-Notebook: /home/sowmya/os_practice$ ls
compare_files.sh  Err  fileA.txt

=====
CASE 2 : WHEN FILES ARE NOT IDENTICAL
=====

sowmya@sowmya-HP-Notebook: /home/sowmya/os_practice$ cat > fileB.txt
This is Operating systems Lab.
This is file B. (Different line)
^C
sowmya@sowmya-HP-Notebook: /home/sowmya/os_practice$ ./compare_files.sh fileA.txt fileB.txt
Files are not identical. No action performed.
Contents of file fileA.txt are:
This is Operating systems Lab.
This is file A.
Contents of file fileB.txt are:
This is Operating systems Lab.
This is file B. (Different line)

=====
ERR FILE CONTENT (if any)
=====

sowmya@sowmya-HP-Notebook: /home/sowmya/os_practice$ cat Err
# (empty - no cmp error)
```

## 5. Write a shell program that takes a command line argument and reports on whether it is directory, a file, or something else.

### File Existence and type tests

| Operator | Description                                                                | Example                     |
|----------|----------------------------------------------------------------------------|-----------------------------|
| -e       | True if the file exists (any type)                                         | [ -e file.txt ]             |
| -f       | True if it exists and is a regular file                                    | [ -f file.txt ]             |
| -d       | True if it exists and is a directory                                       | [ -d /home/user ]           |
| -L / -h  | True if it exists and is a symbolic (soft) link                            | [ -L linkfile ]             |
| -b       | True if it exists and is a block special file (like hard drive)            | [ -b /dev/sda ]             |
| -c       | True if it exists and is a character special file (like keyboard/terminal) | [ -c /dev/tty ]             |
| -p       | True if it is a named pipe (FIFO)                                          | [ -p mypipe ]               |
| -S       | True if it is a socket file                                                | [ -S /var/run/docker.sock ] |

### Algorithm:

1. Start the program.
2. Check if **exactly one argument** is passed. If not, display usage and exit.
3. Store the argument in a variable.
4. Check the type if:
  - o **f** → print “It is a regular file.”
  - o **d** → print “It is a directory.”
  - o **c** → print “It is Character Device file.”
  - o **b** → print “It is Block Device file.”
  - o **L** → print “It is link file
  - o **Else** → print “unrecognized”
5. Stop the program.

### Test Cases:

1. Input a.txt
2. Input any directory name in the current directory
3. Input any file name which is not in the current directory

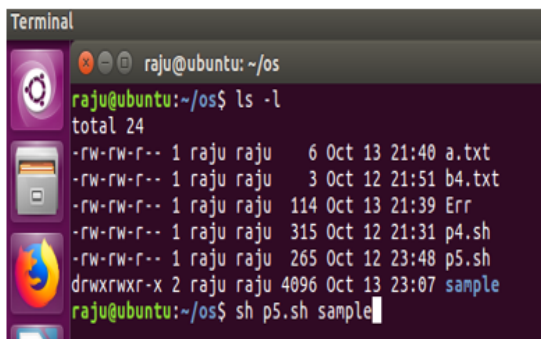
4. Change the directory to /dev
  - a. Input tty
  - b. Input fd0
  - c. Input stdin

**Program:**

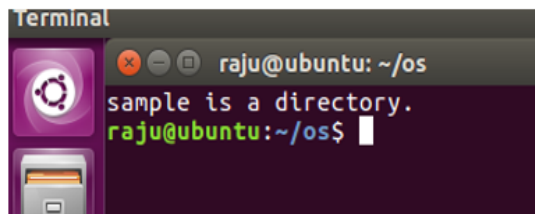
```
clear
if [ -f $1 ]
then
    echo $1 is a regular file.
elif [ -d $1 ]
then
    echo $1 is a directory.
elif [ -L $1 ]
then
    echo $1 is a link file.
elif [ -c $1 ]
then
    echo $1 is a character device.
elif [ -b $1 ]
then
    echo $1 is a block device.
else
    echo $1 is unrecognized.
fi
```

Output:

#\$p5.sh FileName or PathName

**Output:**

```
Terminal
raju@ubuntu: ~/os
raju@ubuntu:~/os$ ls -l
total 24
-rw-rw-r-- 1 raju raju  6 Oct 13 21:40 a.txt
-rw-rw-r-- 1 raju raju  3 Oct 12 21:51 b4.txt
-rw-rw-r-- 1 raju raju 114 Oct 13 21:39 Err
-rw-rw-r-- 1 raju raju 315 Oct 12 21:31 p4.sh
-rw-rw-r-- 1 raju raju 265 Oct 12 23:48 p5.sh
drwxrwxr-x 2 raju raju 4096 Oct 13 23:07 sample
raju@ubuntu:~/os$ sh p5.sh sample
```



```
Terminal
raju@ubuntu: ~/os
sample is a directory.
raju@ubuntu:~/os$
```

6. **Write a shell script that accepts two file names as arguments checks if the permission for these files are identical and if the permissions are identical, output common permission and otherwise output each file name followed by its permission.**

#chmod used for changing the file permission

**usage:** chmod user(s) operation permission(s)

#user - u user, g group, o others, a all, operation + or - , permission- r(read), w (write) and x (Execute)

Eg: chmod u+x p6.sh (or) chmod 777 p6.sh

### **Algorithm**

**Step 1:** Start

**Step 2:** Clear the screen

**Step 3:** Input two filenames — file1 and file2 as command line arguments

**Step 4:** Retrieve the permissions of the first file (file1)

4.1 Display the list of files

4.2 cut the characters from 2 to 10 from the output of step 4.1

4.3 .Store the result in variable p1

**Step 5:** Retrieve the permissions of the second file (file2)

5.1 Display the list of files

5.2 cut the characters from 2 to 10 from the output of step 4.1

5.3 .Store the result in variable p2

**Step 6:** Compare the two permission strings (p1 and p2)

If p1 = p2 then

Print "File permissions of file1 and file2 are identical."

Print the common permission value (p1)

Else

Print "File permissions of file1 and file2 are not identical."

Print "file1 permissions are: p1"

Print "file2 permissions are: p2"

End If

**Step 7:** Stop

Test Case 1:

1. Display the list of files in the current directory with access permissions
2. Take input of two files with same access permission and give as command line arguments and observe the output.

Test Case 2:

3. Change the access permission of b.txt (include executable permission to all users)

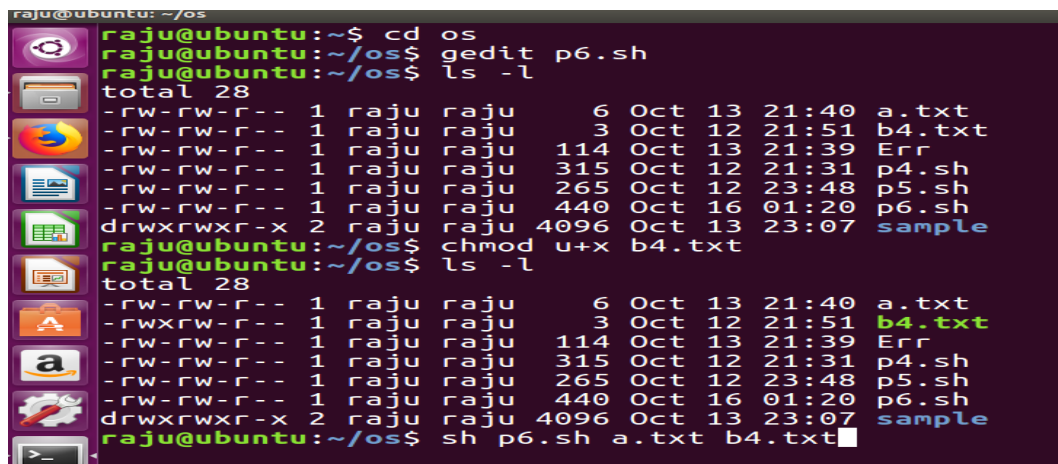


4. Execute the program with 2 files( different access permission) as command line arguments and observe the output.

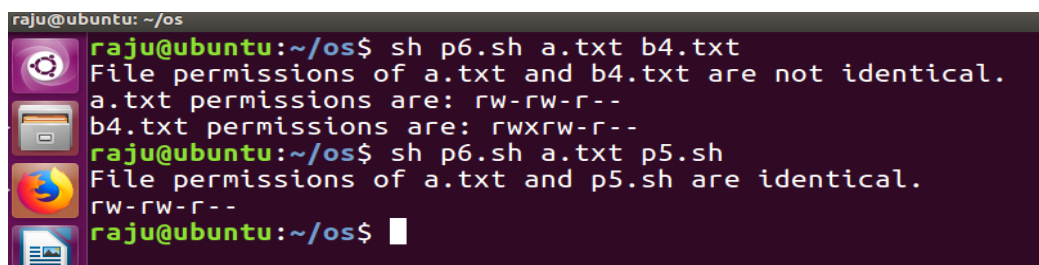
### Program:

```
clear
# select and Store the permissions of the first(file) argument
p1=`ls -l $1 | cut -c 2-10`
# select Store the permissions of the second (file) argument.
p2=`ls -l $2 | cut -c 2-10`
if [ "$p1" = "$p2" ]
then
echo File permissions of $1 and $2 are identical.
echo $p1 # Print the common permission of both files.
else
echo File permissions of $1 and $2 are not identical.
echo "$1 permissions are: $p1"
echo "$2 permissions are: $p2"
fi
```

### Output:



```
raju@ubuntu: ~/os
raju@ubuntu:~$ cd os
raju@ubuntu:~/os$ gedit p6.sh
raju@ubuntu:~/os$ ls -l
total 28
-rw-rw-r-- 1 raju raju 6 Oct 13 21:40 a.txt
-rw-rw-r-- 1 raju raju 3 Oct 12 21:51 b4.txt
-rw-rw-r-- 1 raju raju 114 Oct 13 21:39 Err
-rw-rw-r-- 1 raju raju 315 Oct 12 21:31 p4.sh
-rw-rw-r-- 1 raju raju 265 Oct 12 23:48 p5.sh
-rw-rw-r-- 1 raju raju 440 Oct 16 01:20 p6.sh
drwxrwxr-x 2 raju raju 4096 Oct 13 23:07 sample
raju@ubuntu:~/os$ chmod u+x b4.txt
raju@ubuntu:~/os$ ls -l
total 28
-rw-rw-r-- 1 raju raju 6 Oct 13 21:40 a.txt
-rwxrw-r-- 1 raju raju 3 Oct 12 21:51 b4.txt
-rw-rw-r-- 1 raju raju 114 Oct 13 21:39 Err
-rw-rw-r-- 1 raju raju 315 Oct 12 21:31 p4.sh
-rw-rw-r-- 1 raju raju 265 Oct 12 23:48 p5.sh
-rw-rw-r-- 1 raju raju 440 Oct 16 01:20 p6.sh
drwxrwxr-x 2 raju raju 4096 Oct 13 23:07 sample
raju@ubuntu:~/os$ sh p6.sh a.txt b4.txt
```



```
raju@ubuntu: ~/os
raju@ubuntu:~/os$ sh p6.sh a.txt b4.txt
File permissions of a.txt and b4.txt are not identical.
a.txt permissions are: rw-rw-r--
b4.txt permissions are: rwxrw-r--
raju@ubuntu:~/os$ sh p6.sh a.txt p5.sh
File permissions of a.txt and p5.sh are identical.
rw-rw-r--
raju@ubuntu:~/os$
```

## 7. Write a shell program to print the arguments in reverse order

A **for loop** in Bash is a **control structure** used to **repeat a set of commands multiple times**.

- It iterates over a **list of items** or performs **arithmetic repetition**.
- Useful for automating repetitive tasks.

### C style for loop (arithmetic- based)

**for (( INITIALIZATION; CONDITION; INCREMENT/DECREMENT ))**

**do**

**commands**

**done**

### **Special Parameters**

**\$\*** Represents all of the command-line arguments.

**\$#** Contains the number of arguments on the command line.

**\$?** Exit status of last command.

**\$\$** PID of the current shell.

**\$\_** PID of the last back ground job.

### **Algorithm:**

1. Start the program.
2. Check if at least **one argument** is provided.
  - If not, display usage and exit.
3. Loop from the **last argument** to the **first**.
4. Print each argument in the loop.
5. End the program.

### **Program:**

```
echo Given arguments are: $*
```

```
# Step 1: Check if at least one argument is provided
```

```
if [ $# -eq 0 ]; then
```

```
    echo "Usage: $0 arg1 arg2 ..."
```

```
    exit 1
```

```
fi
```

```
# Step 2: Loop from last argument to first
```

```
for (( i=$#; i>0; i-- ))
```

```
do
```

```
    echo "${!i}"    #Indirect expansion ${!i} is used to access a variable dynamically (ith argument)
```

```
done
```

**Output:**

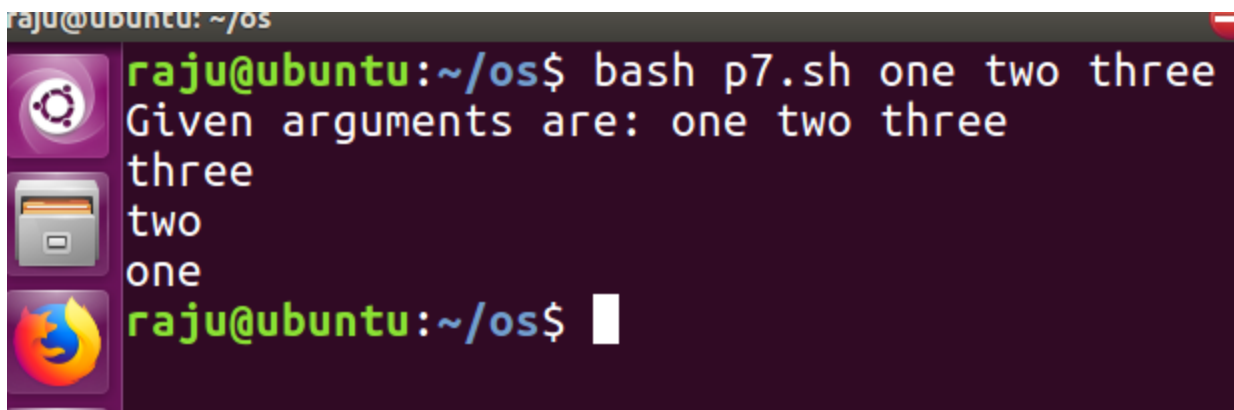
Test Case 1:

The output should be

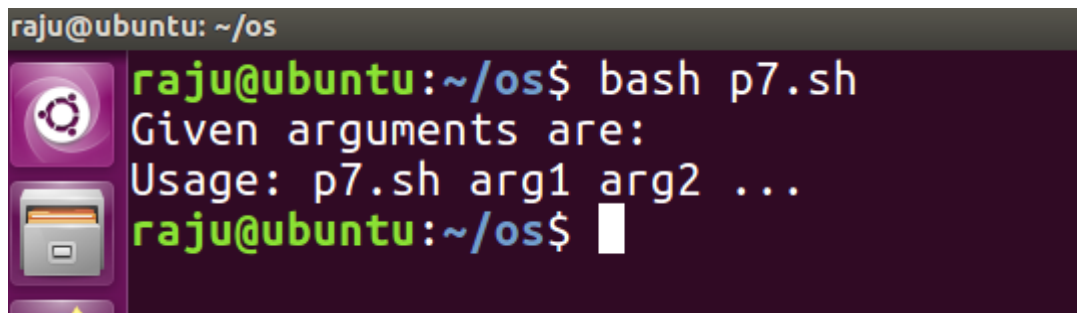
```
bash p7.sh one two three
three
two
one
```

Test Case 2:

Give Input with only one argument and observe the output



```
raju@ubuntu: ~/os
raju@ubuntu:~/os$ bash p7.sh one two three
Given arguments are: one two three
three
two
one
raju@ubuntu:~/os$
```



```
raju@ubuntu: ~/os
raju@ubuntu:~/os$ bash p7.sh
Given arguments are:
Usage: p7.sh arg1 arg2 ...
raju@ubuntu:~/os$
```

## 8. Write a shell program to print the prime numbers between the specified range

### Algorithm:

**Step 1:** Start the program.

**Step 2:** Display a message asking the user to enter the lower limit.

→ Read the value into the variable lower.

**Step 3:** Display a message asking the user to enter the upper limit.

→ Read the value into the variable upper.

**Step 4:** Display a message:

“Prime numbers between *lower* and *upper* are:”

**Step 5:** Repeat steps 6–10 for each number num from lower to upper:

**Step 5.1:** If num < 2, skip to the next number (since 0 and 1 are not prime).

**Step 5.2:** Assume the number is prime (set a flag is\_prime = 1).

**Step 5.3 :** For each integer i from 2 to square root of num:

    If num % i == 0, then

    → Set is\_prime = 0 (number is not prime).

    → Break the inner loop.

**Step 5.4:** If is\_prime == 1, print the value of num (it is prime).

**Step 5.5:** Continue the loop for the next number.

**Step 6:** After all numbers are checked, stop the loop.

**Step 7:** End the program.

### **Test Case :**

1. Enter the lower limit and upper limit as 10 and 20
2. Enter the lower limit and upper limit as -2 and 30

### Program:

*# Prompt the user for the lower and upper limits of the range*

echo "Enter the lower limit of the range:"

read lower

echo "Enter the upper limit of the range:"

read upper

echo "Prime numbers between \$lower\_limit and \$upper\_limit are:"

*# Iterate through each number in the specified range*

for (( num = lower; num <= upper; num++ ));

do

*# 0 and 1 are not prime numbers*

    if [ \$num -lt 2 ]; then

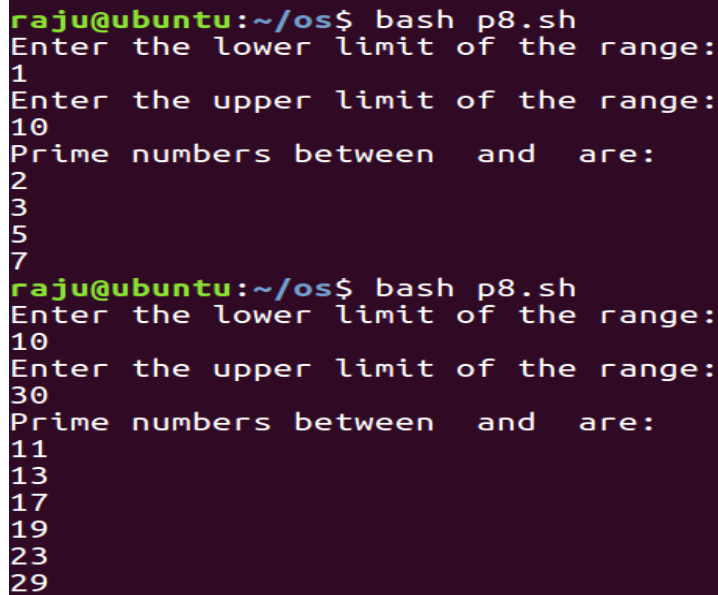
        continue

    fi

    is\_prime=1 *# Assume the number is prime initially*

```
# Check for divisibility from 2 up to the square root of the number
# Optimizing by checking up to num/2 or sqrt(num) is more efficient

for (( i = 2; i * i <= num; i++ ))
do
    if [ $((num%i)) -eq 0 ]; then
        is_prime=0 # Not a prime number
        break
    fi
done
# If is_prime is still 1, then the number is prime
if [ $is_prime -eq 1 ]; then
    echo "$num"
fi
done
```

**Output:**

```
raju@ubuntu:~/os$ bash p8.sh
Enter the lower limit of the range:
1
Enter the upper limit of the range:
10
Prime numbers between 1 and 10 are:
2
3
5
7

raju@ubuntu:~/os$ bash p8.sh
Enter the lower limit of the range:
10
Enter the upper limit of the range:
30
Prime numbers between 10 and 30 are:
11
13
17
19
23
29
```

## PART-C

### 1. Write a C program to implement UNIX ln, mv, rm commands using APIs.

#### Case1: Implementing ln (Create Hard Link)

##### Algorithm:

1. Start.
2. Accept two filenames — source file (oldpath) and link name (newpath).
3. Call the system call link(oldpath, newpath).
4. If the call returns 0 → success, else print an error using perror().
5. Stop.

##### Test Case1:

Create Links in a Directory (-t Option)

mkdir links

ln -t links source.txt

##### Operation:

Creates a **hard link** to source.txt inside the directory links/ with the same filename.

##### Verification:

List the directory contents:

ls -l links/

##### Purpose:

To verify that the -t option correctly places the link in the specified target directory.

Create Multiple Links in One Command

mkdir multiple\_links

ln source.txt file1.txt file2.txt multiple\_links/

##### Operation:

Creates **hard links** to source.txt with the names file1.txt and file2.txt inside the directory multiple\_links/.

##### Program:

```
#include <stdio.h>
```

```
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <existing_file> <new_link>\n", argv[0]);
        return 1;
    }
    if (link(argv[1], argv[2]) == 0)
        printf("Hard link created successfully.\n");
    else
        perror("Error creating hard link");
    return 0;
}
```

**Expected Output:**

Multiple hard links created successfully in directory 'multiple\_links/'.

**Purpose:**

To verify that ln supports creation of **multiple hard links** in a single command when a target directory is specified.

**Case 2. Implementing mv (Move / Rename File)****Algorithm:**

1. Start.
2. Accept two filenames — source (oldpath) and destination (newpath).
3. Call the system call rename(oldpath, newpath).
4. If it returns 0 → print success, else show error using perror().
5. Stop.

**Program:**

```
#include <stdio.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <source_file> <destination_file>\n", argv[0]);
        return 1;
    }
}
```

```
if (rename(argv[1], argv[2]) == 0)
    printf("File moved/renamed successfully.\n");
else
    perror("Error moving/renaming file");
return 0;
}
```

**Output:****Test Case2:**

Command: mv

- Input: oldname.txt newname.txt
- Operation: Moves or renames oldname.txt to newname.txt using the rename() system call.
- Expected Output: Displays the message  
*"File renamed/moved successfully."*
- Purpose: To check whether the file can be successfully renamed or moved.

**Case 3. Implementing rm (Remove File)****Algorithm:**

1. Start.
2. Accept a filename to remove.
3. Call the system call unlink(filename).
4. If it returns 0 → success message, else show error using perror().
5. Stop.

**Program:**

```
#include <stdio.h>
#include <unistd.h>
int main(int argc, char *argv[])
{
    if (argc != 2) {
        printf("Usage: %s <file_to_remove>\n", argv[0]);
        return 1;
    }
    if (unlink(argv[1]) == 0)
```



```
    printf("File removed successfully.\n");  
else  
    perror("Error removing file");  
return 0;  
}
```

**Output:**

| Test Case:3 Remove a Directory |
|--------------------------------|
|--------------------------------|

- |                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ol style="list-style-type: none"><li>1. Keep some files in the directory and try to remove it and observe the output.</li><li>2. Remove all the files in the directory and try to remove it and observe the output.</li></ol> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**2. Write a program in C to display the contents of a named file on standard output device., Also Write a program to copy the contents of one file to another.**

**Part 1: Display the contents of a file on standard output (like cat command)**

**Algorithm:**

1. Start
2. Accept the filename as a command-line argument.
3. Open the file in **read mode** using fopen() (or open() system call).
4. If the file cannot be opened, display an error and exit.
5. Read the file character by character using fgetc() (or byte by byte).
6. Print each character to the standard output using putchar().
7. Close the file.
8. Stop.

**Program:**

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    FILE *fp;
    int ch;
    if (argc != 2) {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1;
    }
    fp = fopen(argv[1], "r");
    if (fp == NULL) {
        perror("Error opening file");
        return 1;
    }

    // Read and display contents
    while ((ch = fgetc(fp)) != EOF) {
        putchar(ch);
    }
    fclose(fp);
    return 0;
}
```

## **Output:**

### **Test Case 1:**

- **Input:** An empty file — empty.txt
- **Operation:** Open and read the file.
- **Expected Output:** No content displayed (blank output), but no error message shown.
- **Purpose:** To verify the handling of empty files without runtime errors.

### **Test Case 2:**

- **Input:** A large file — bigfile.txt (e.g., several KBs of text)
- **Operation:** Display the file contents completely on standard output.
- **Expected Output:** The entire file content is printed correctly on the screen without truncation.
- **Purpose:** To ensure the program handles large file data properly.

## **Part 2: Copy contents of one file to another (like cp command)**

### **Algorithm:**

1. Start
2. Accept two filenames — source file and destination file.
3. Open the source file in **read mode** and destination file in **write mode**.
4. If either file cannot be opened, display an error and exit.
5. Read each character from the source file and write it to the destination using `fgetc()` and `fputc()`.
6. After copying, close both files.
7. Display a success message.
8. Stop.

### **Program:**

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    FILE *src, *dest;
    int ch;
    if (argc != 3) {
        printf("Usage: %s <source_file> <destination_file>\n", argv[0]);
```

```
    return 1;
}
src = fopen(argv[1], "r");
if (src == NULL) {
    perror("Error opening source file");
    return 1;
}
dest = fopen(argv[2], "w");
if (dest == NULL) {
    perror("Error opening destination file");
    fclose(src);
    return 1;
}
// Copy contents
while ((ch = fgetc(src)) != EOF) {
    fputc(ch, dest);
}
printf("File copied successfully.\n");
fclose(src);
fclose(dest);
return 0;
}
```

## **Output:**

|                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Test Case1:</b>                                                                                                                                                                                                                                                                                                                                       |
| <ul style="list-style-type: none"><li>• <b>Input:</b> Source file — empty.txt; Destination file — copy.txt</li><li>• <b>Operation:</b> Copy contents from an empty source file.</li><li>• <b>Expected Output:</b> copy.txt is created but remains empty.</li><li>• <b>Purpose:</b> To confirm correct behavior when copying empty files.</li></ul>       |
| <b>Test Case 2:</b>                                                                                                                                                                                                                                                                                                                                      |
| <ul style="list-style-type: none"><li>• <b>Input:</b> Source file — source.txt; Destination file — a read-only file readonly.txt</li><li>• <b>Operation:</b> Attempt to write contents into a file that does not have write permission.</li><li>• <b>Expected Output:</b> Error message — “Error opening destination file: Permission denied.”</li></ul> |
| <b>Purpose:</b> To check the program’s response when file writing permissions are restricted.                                                                                                                                                                                                                                                            |

### 3. Write a C program that reads every 100th byte from the file, where the file name is given as command line argument.

#### Algorithm: Read every 100th byte from a file

1. Start
2. Accept the filename as a command-line argument.
3. Open the file in read mode ("r").
4. If the file cannot be opened, display an error and exit.
5. Initialize a counter pos = 0.
  - Use a loop to:
  - Move the file pointer to the pos byte using fseek(file, pos, SEEK\_SET).
  - Read one byte using fgetc(file).
  - If end-of-file is reached, break the loop.
  - Print the byte (as a character).
  - Increment pos by 100 (pos += 100)
7. Close the file.
8. Stop

#### **Program:**

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    FILE *fp;
    int ch;
    long pos = 0;

    if (argc != 2) {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1;
    }
    fp = fopen(argv[1], "r");
    if (fp == NULL) {
        perror("Error opening file");
        return 1;
    }
    printf("Every 100th byte from the file '%s':\n", argv[1]);
    while (1)
    {
```

```
if (fseek(fp, pos, SEEK_SET) != 0) // move file pointer
    break;

ch = fgetc(fp); // read one byte
if (ch == EOF)
    break;

printf("Byte %ld: %c\n", pos, ch);
pos += 100; // move to next 100th byte
}

fclose(fp);
return 0;
}
```

### **Output:**

| Test Case 1:                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• <b>Input:</b> A valid text file name — sample.txt (contains more than 200 bytes).</li><li>• <b>Operation:</b> The program opens sample.txt and reads every 100th byte using fseek() and fgetc().</li><li>• <b>Expected Output:</b><br/>The program prints bytes at positions 0, 100, 200, 300, ... until EOF, for example:</li></ul> |
| Every 100th byte from the file 'sample.txt':<br>Byte 0: H<br>Byte 100: e<br>Byte 200: l<br>Byte 300: o<br><b>Purpose:</b> To verify that the program correctly reads and displays every 100th byte of a valid file.                                                                                                                                                          |

## 4. Write a C program to create a process by using fork () and vfork() system call.

### 1. Using fork() System Call:

#### Algorithm:

Start

Call the system call fork().

Check the return value of fork():

If it returns 0, it's the child process → print a message and child's PID.

- If it returns > 0, it's the parent process → print a message and parent's PID.
- If it returns < 0, the fork failed → print an error message.

Both processes continue execution independently.

Stop.

#### Program:

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork(); // Create a new process
    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process: PID = %d, Parent PID = %d\n", getpid(), getppid());
    }
    else {
        // Parent process
        printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);
    }
    printf("Process %d is executing this line.\n", getpid());
    return 0;
}
```

#### Output:

**Test Case 1:**

Input: Run the program with no arguments.

Operation: The program calls fork() to create a child process.

Expected Output:

The output should display two messages — one from the parent and one from the child process.

Purpose: To verify that the fork() system call successfully creates a child process and both processes execute independently.

**2. Using vfork() System Call:****Algorithm:**

1. **Start**
2. Call the system call vfork().
3. Check the return value:
  - If **0** → this is the **child process**, execute child code and **call exit() or exec() immediately**.
  - If **> 0** → this is the **parent process**, wait for the child to complete before continuing.
4. If **< 0** → print an error message.

**Stop.**

**Program:**

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main()
{
    pid_t pid;
    pid = vfork(); // Create a new process sharing address space
    if (pid < 0) {
        perror("vfork failed");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process (vfork): PID = %d, Parent PID = %d\n", getpid(), getppid());
        _exit(0); // Use _exit() instead of return or exit() to prevent corruption
    }
}
```



```
}  
else {  
    // Parent process resumes after child exits  
    printf("Parent Process (vfork): PID = %d, Child PID = %d\n", getpid(), pid);  
}  
return 0;  
}
```

### **Output:**

| Test Case 1:                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• <b>Input:</b> Run the program with no arguments.</li><li>• <b>Operation:</b> The program calls vfork() to create a child process that shares the parent's address space.</li><li>• <b>Expected Output:</b><br/>The child process runs first, then the parent resumes after child calls _exit(0).</li><li>• <b>Purpose:</b> To verify that the child process runs before the parent resumes execution, as per vfork() behavior.</li></ul> |
| Test Case 2:                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <ul style="list-style-type: none"><li>• Input: None</li><li>• Operation: Modify the program to include a print statement after _exit(0) in the child.</li><li>• Expected Output:<br/>No message should appear after _exit(0) since _exit() terminates immediately.</li><li>• Purpose: To ensure _exit() properly terminates the child without executing further statements or corrupting shared memory.</li></ul>                                                                |
| Test Case 3:                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <ul style="list-style-type: none"><li>• Input: Simulate system load (many processes).</li><li>• Operation: Run the program under resource constraints.</li><li>• Expected Output:<br/>If process creation fails, it prints:<br/>vfork failed: Resource temporarily unavailable</li><li>• Purpose: To verify that vfork() handles failure conditions gracefully.</li></ul>                                                                                                        |

## 5. Write a C program to demonstrate the process is Zombie and to avoid the Zombie process.

### Algorithm to Create a Zombie Process

1. Start
2. Call fork() to create a child process.
3. If fork() < 0 → print error and exit.
4. If in child process:
  - Print child PID.
  - Exit immediately using exit(0).
5. If in parent process:
  - Print parent PID.
  - Do not call wait(); instead, call sleep() for some time.
  - During this time, the child has already exited → becomes a Zombie.
6. After sleep, the parent exits → Zombie is cleared.
7. Stop.

### Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("Child process created. PID = %d\n", getpid());
        printf("Child exiting...\n");
    }
```

```
        exit(0); // Child terminates
    }
    else {
        // Parent process
        printf("Parent process. PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent sleeping... Check zombie using 'ps -l'\n");
        sleep(20); // Keep parent alive, child becomes zombie
        printf("Parent exiting now.\n");
    }
    return 0;
}
```

**Output:****Test Case 1:**

- Input: Run the program with no arguments.
- Operation: The program calls fork() to create a child process. The child prints its PID and exits immediately using exit(0), while the parent sleeps for 20 seconds.
- Expected Output:  
On terminal:

During the sleep period, execute the command ps -l in another terminal.

The output will show the child process with state 'Z' (Zombie).

Purpose: To verify that when the parent does not call wait(), the terminated child process becomes a Zombie until the parent exits.

**2. Avoiding Zombie Process:****Algorithm to Avoid Zombie Process**

1. Start
2. Create a child process using fork().
3. If fork() < 0 → error.

If in child process:

- Print PID.
- Perform task and exit.

If in parent process:

- Call wait() to collect child's termination status.
- Print confirmation that the child has ended.

Stop.

### **Program:**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid;
    int status;
    pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0)
    {
        // Child process
        printf("Child process running. PID = %d\n", getpid());
        sleep(2);
        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        // Parent process
        printf("Parent process. PID = %d\n", getpid());
        wait(&status); // Parent waits for child
        printf("Child has terminated. No zombie created.\n");
    }
    return 0;
}
```

**Output:****Test Case 1:**

- **Input:** Run the program with no arguments.

- **Operation:**

1. The parent process creates a child using fork().
2. The child process prints its PID, runs for 2 seconds, then exits using exit(0).
3. The parent waits for the child using wait(&status)

- **Purpose:**

To verify that using wait() in the parent process prevents the creation of a Zombie process by properly collecting the child's exit status.

## 6. Write a C program to create an Orphan Process.

### Algorithm to Create an Orphan Process

1. Start
2. Call fork() to create a child process.
3. Check the return value of fork():
  - If  $< 0$   $\rightarrow$  fork failed  $\rightarrow$  print error and exit.
  - If  $= 0$   $\rightarrow$  child process:
    - Print its PID and parent PID.
    - Sleep for some time (e.g., 5–10 seconds) to allow parent to terminate.
    - Print its parent PID again after parent exits (it should be 1, the init process).
4. If  $> 0$   $\rightarrow$  parent process:
  - Print parent PID.
  - Exit immediately, making the child process an orphan.
5. Stop.

### **Program:**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    pid_t pid;
    pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0)
    {
        // Child process
        printf("Child process started. PID = %d, Parent PID = %d\n", getpid(), getppid());
        sleep(5); // Give time for parent to terminate
        printf("Child process running after parent exit. New Parent PID = %d\n", getppid());
    }
    else
    {
        // Parent process
        printf("Parent process. PID = %d\n", getpid());
    }
}
```

```
    printf("Parent exiting immediately to create orphan child.\n");
    exit(0); // Parent exits immediately
}
return 0;
}
```

### **Output:**

#### **Test Case 1: Creation of an Orphan Process**

##### **Input:**

Run the program without any command-line arguments.

##### **Operation Steps:**

1. The parent process calls fork () to create a child process.
2. The child process prints its PID and its parent's PID.
3. The parent process prints its PID and exits immediately, making the child process orphaned.
4. The child process sleeps for 5 seconds.
5. After the parent exits, the child prints its new parent PID (which should now be **1**, the init or systemd process).

##### **Purpose:**

To verify that when a parent process terminates before its child, the child becomes an **orphan process**, and its parent PID changes to **1** (adopted by init/systemd).

## 7. Write a C program to demonstrate a parent process that uses wait() system call to catch the child's exit code.

### Prerequisites

The **fork()** system call is used to create a child process.

The **wait()** system call makes the parent suspend execution until one of its child processes terminates.

Once the child finishes, **wait()** returns the **PID** of the terminated child and stores its exit status in an integer variable passed by reference.

This exit code can be examined using macros:

**WIFEXITED(status)** – true if the child exited normally.

**WEXITSTATUS(status)** – returns the child's exit code.

### Algorithm

1. Start the program.
2. Call fork() to create a child process.
3. If fork() returns 0, execution is in the child process:
  - a. Print the child's PID.
  - b. Use exit(code) to terminate with an explicit exit status (for example, exit(5)).
4. If fork() returns a positive value, execution is in the parent:
  - a. Call wait(&status) to wait for the child to terminate.
  - b. When wait() returns, extract the exit code using:

if (WIFEXITED(status))

printf("Exit code: %d", WEXITSTATUS(status));

5. Display both process IDs and the child's exit code.

6. Stop the program

### Program:

// Program: Demonstrate parent using wait() to catch child exit codes

// Test Case: Two children exiting with different codes (3 and 7)

```
#include <stdio.h>
#include <stdlib.h> // for exit()
#include <unistd.h> // for fork(), getpid()
#include <sys/types.h> // for pid_t
```



```
#include <sys/wait.h> // for wait(), waitpid(), macros

int main(void)
{
    pid_t pid1, pid2;
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    // ----- Create first child -----
    pid1 = fork();

    if (pid1 < 0) {
        perror("fork() for child 1 failed");
        exit(1);
    }

    if (pid1 == 0) {
        // Child 1 code
        printf("Child 1 process running. PID = %d\n", getpid());
        // Exit with code 3
        exit(3);
    }

    // ----- Create second child (executed only in parent) -----
    pid2 = fork();

    if (pid2 < 0) {
        perror("fork() for child 2 failed");
        exit(1);
    }

    if (pid2 == 0) {
        // Child 2 code
        printf("Child 2 process running. PID = %d\n", getpid());
        // Exit with code 7
        exit(7);
    }

    // ----- Parent process waits for both children -----
    printf("Parent waiting for children...\n");

    // Wait for child 1
    if (waitpid(pid1, &status, 0) == -1) {
        perror("waitpid() for child 1 failed");
    } else {
        if (WIFEXITED(status)) {
```

```

        printf("Child with PID %d exited normally with exit code: %d\n",
            pid1, WEXITSTATUS(status));
    } else {
        printf("Child with PID %d did not exit normally.\n", pid1);
    }
}

// Wait for child 2
if (waitpid(pid2, &status, 0) == -1) {
    perror("waitpid() for child 2 failed");
} else {
    if (WIFEXITED(status)) {
        printf("Child with PID %d exited normally with exit code: %d\n",
            pid2, WEXITSTATUS(status));
    } else {
        printf("Child with PID %d did not exit normally.\n", pid2);
    }
}

printf("Parent process (PID = %d) finished.\n", getpid());

return 0;
}

```

### **Output:**

#### **Test Case 1: Multiple Children (Parent Waits for Each)**

- Input:  
Modify the program to call fork() twice to create two children.
- Operation Steps:
  1. Both children print their PIDs and exit with different codes (e.g., 3 and 7).
  2. Parent waits twice (once for each child) and displays their exit codes.

#### • Expected Output:

```

Child 1 process running. PID = 5501
Child 2 process running. PID = 5502
Parent waiting for children...
Child with PID 5501 exited normally with exit code: 3
Child with PID 5502 exited normally with exit code: 7

```

## 8. Write a program to demonstrate race conditions.

### Prerequisites

A race condition occurs when multiple processes or threads execute concurrently and the final output depends on the order in which the processes are scheduled by the operating system. In such cases, the processes “race” to access a shared resource, causing inconsistent or incorrect results.

In UNIX systems, the fork() call can lead to race conditions when both parent and child processes attempt to print or access shared data without synchronization. The execution order between the two can't be predicted.

### Algorithm

1. Start the program.
2. Create a new child process using fork().
3. After the fork:
  - a. The child process runs a function that prints a message character by character.
  - b. The parent process runs the same function with a different message.
4. Because of the lack of synchronization, both parent and child may print simultaneously, causing characters to interleave randomly.
5. Run the program multiple times to observe different print orders due to the race condition.

### Program:

// Program: Demonstrate race condition using fork() and shared stdout

// Parent and child both print messages character-by-character without synchronization.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // fork(), getpid(), usleep()
#include <sys/types.h> // pid_t
#include <sys/wait.h> // wait()

void charatime(const char *str)
{
    const char *p;

    for (p = str; *p != '\0'; p++) {
        putchar(*p);    // print one character at a time
    }
}
```

```
        fflush(stdout);    // force flush so it appears immediately
        usleep(1000);     // small delay to increase chance of interleaving
    }
}

int main(void)
{
    pid_t pid;

    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // Child process
        printf("Child PID: %d (started)\n", getpid());
        charatime("Output from child\n");
        exit(0);
    } else {
        // Parent process
        printf("Parent (PID: %d) continues after fork. Child PID: %d\n",
            getpid(), pid);
        charatime("Output from parent\n");

        // Wait for child to finish (does NOT prevent race on stdout,
        // because both were already running concurrently)
        wait(NULL);
    }

    return 0;
}
```

### **Output:**

| Test Case 1: Demonstrate Interleaving Output Due to Race Condition                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• <b>Input:</b><br/>Run the program normally (no command-line arguments).</li><li>• <b>Operation Steps:</b><ol style="list-style-type: none"><li>1. The program creates a child process using fork().</li></ol></li></ul> |

2. Both the **parent** and **child** processes call the `charatime()` function to print different strings:

- Parent: "Output from parent\n"
- Child: "Output from child\n"

3. Since there is **no synchronization**, both processes may print simultaneously.

4. Run the program multiple times to observe the **interleaving of characters**.

**Expected Output (Sample 1):**

Output from parent

Output from child

**Expected Output (Sample 2 — Interleaved):**

OutOuptupt u ft rfromom cphialrdent

*(Characters from both processes are mixed randomly, depending on CPU scheduling.)*

**Purpose:**

To demonstrate a **race condition** where two concurrent processes access the same output stream (stdout) without synchronization, causing **unpredictable and inconsistent output**.

**9.write a program to create, writes to, and reads from a pipe. Also write a program to create a pipe from the parent to child and send data down the pipe.**

### **Prerequisites**

The **pipe()** system call creates two file descriptors:

fd → read end of the pipe

fd[1] → write end of the pipe

The same process writes to and reads from the pipe sequentially.

Pipes are unidirectional, hence only one direction of data flow per pipe.

### **1. Algorithm to create, writes to, and reads from a pipe within the same process:**

1. Create a pipe using pipe(fd).
2. If creation fails, print an error message.
3. Write a message into the write-end fd[1].
4. Read from the read-end fd[0].
5. Print the message read through the pipe

### **Program:**

// Program 1: Create a pipe, write to it, and read from it in the same process

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // pipe(), read(), write()
#include <string.h> // strlen()

int main() {
    int fd[2]; // fd[0] = read end, fd[1] = write end
    char write_msg[] = "Hello from the pipe!";
    char read_buffer[100];

    // 1. Create a pipe
    if (pipe(fd) == -1) {
        perror("Pipe creation failed");
        exit(1);
    }

    // 2. Write to the pipe
    write(fd[1], write_msg, strlen(write_msg) + 1);
```

```
// 3. Read back from the pipe
read(fd[0], read_buffer, sizeof(read_buffer));

// 4. Print the data read
printf("Message read from pipe: %s\n", read_buffer);

return 0;
}
```

## **2. Algorithm to create a pipe from the parent to child and send data down the pipe:**

1. Create a pipe using pipe(fd).
2. Use fork() to create a child process.
3. In the parent process: Write a message to the write-end (fd[1]).
4. In the child process: Read that message from the read-end (fd[0]).
5. Display the received message in the child process.

### **Explanation:**

The pipe enables one-way communication between parent and child. File descriptors are shared after fork(). The parent closes the read end; the child closes the write end. The message written by the parent is read by the child through the pipe.

### **Program:**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    pid_t pid;
    char message[] = "Hello Child, this is Parent!";
    char buffer[100];

    // 1. Create a pipe
    if (pipe(fd) == -1) {
```

```
perror("Pipe creation failed");
exit(1);
}

// 2. Create a child process
pid = fork();

if (pid < 0) {
    perror("Fork failed");
    exit(1);
}

if (pid == 0) {
    // ----- CHILD PROCESS -----
    close(fd[1]);          // Close write end
    read(fd[0], buffer, sizeof(buffer));
    printf("Child read from pipe: %s\n", buffer);
    close(fd[0]);          // Close read end
}
else {
    // ----- PARENT PROCESS -----
    close(fd[0]);          // Close read end
    write(fd[1], message, strlen(message) + 1);
    printf("Parent wrote to pipe: %s\n", message);
    close(fd[1]);          // Close write end
}

return 0;
}
```



**Output:**

```
sowmya@sowmya-HP-Notebook: ~$ gcc pipe_single_process.c -o pipe1
sowmya@sowmya-HP-Notebook: ~$ ./pipe1
Message read from pipe: Hello from the pipe!

sowmya@sowmya-HP-Notebook: ~$ gcc pipe_parent_child.c -o pipe2
sowmya@sowmya-HP-Notebook: ~$ ./pipe2
Parent wrote to pipe: Hello Child, this is Parent!
Child read from pipe: Hello Child, this is Parent!
```

## 10. Write a program to create a semaphore and initialize value to the semaphore.

- a. create a binary semaphore
- b. create a counting semaphore

### Prerequisites:

- **Binary semaphore:** It can have values 0 or 1. It is often used as a mutex. It works like a lock (can be locked or unlocked).
- **Counting semaphore:** It can have a value greater than 1. It is used to control access to a resource with multiple instances. Initialized with a user-defined value (e.g., 3 in this example), representing the number of available resources
- `sem_init(sem_t *sem, int pshared, unsigned int value):` Initializes the semaphore. pshared is 0 for intra-process use.

### Algorithm:

#### Step-by-Step Algorithm

1. **Start** the program.
2. **Include** the required header files:
  - `stdio.h`, `stdlib.h`, and `semaphore.h`.
3. **Declare** two semaphore variables — one for a **binary semaphore** and another for a **counting semaphore**.
4. **Define a function** `create_binary_semaphore(sem_t *sem)` that:
  - Calls `sem_init(sem, 0, 1)` to initialize the binary semaphore with value 1.
  - If initialization fails, display an error message and exit.
  - If successful, print a confirmation message.
5. **Define another function** `create_counting_semaphore(sem_t *sem, unsigned int value)` that:
  - Calls `sem_init(sem, 0, value)` to initialize the counting semaphore with the given initial value.
  - If initialization fails, display an error message and exit.
  - If successful, print a confirmation message.

**6. In the main() function:**

- Declare sem\_t variables for both binary and counting semaphores.
- Call create\_binary\_semaphore() to initialize the binary semaphore.
- Call create\_counting\_semaphore() with a user-defined initial value (e.g., 3).

7. **Display success messages** for both semaphore initializations.

8. **Stop** the program.

**Program:**

// Program: Create and initialize a binary semaphore and a counting semaphore

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h> // sem_t, sem_init()
#include <errno.h>

// Function to create a binary semaphore (value = 1)
void create_binary_semaphore(sem_t *sem) {
    if (sem_init(sem, 0, 1) == -1) {
        perror("Binary semaphore initialization failed");
        exit(EXIT_FAILURE);
    }
    printf("Binary semaphore created and initialized with value = 1\n");
}

// Function to create a counting semaphore (value = user-defined)
void create_counting_semaphore(sem_t *sem, unsigned int value) {
    if (sem_init(sem, 0, value) == -1) {
        perror("Counting semaphore initialization failed");
        exit(EXIT_FAILURE);
    }
    printf("Counting semaphore created and initialized with value = %u\n", value);
}

int main() {
    sem_t binary_sem;
    sem_t counting_sem;
    unsigned int initial_value = 3; // Example: counting semaphore initial value = 3

    printf("Creating semaphores...\n");

    // Create binary semaphore
    create_binary_semaphore(&binary_sem);
```

```
// Create counting semaphore
create_counting_semaphore(&counting_sem, initial_value);

printf("Both semaphores created successfully.\n");

return 0;
}
```

## **Output:**

```
sowmya@sowmya-HP-Notebook:~$ gcc semaphore_init.c -o sem -pthread
sowmya@sowmya-HP-Notebook:~$ ./sem
```

```
Creating semaphores...
Binary semaphore created and initialized with value = 1
Counting semaphore created and initialized with value = 3
Both semaphores created successfully.
```

---